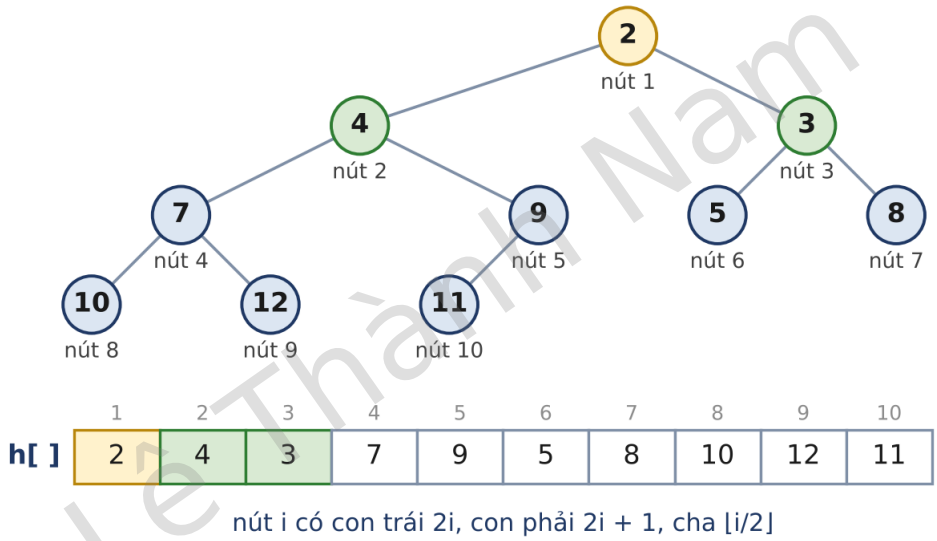
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

CẤU TRÚC DỮ LIỆU DẠNG CÂY

Từ cơ bản đến chuyên sâu

Tập 1. LÝ THUYẾT



nút i có con trái 2i, con phải 2i + 1, cha ⌊i/2⌋

Lê Thành Nam

Chương 1. CÁC CẤU TRÚC CÂY CƠ BẢN
Chương 2. CÂY PHÂN ĐOẠN NÂNG CAO
Chương 3. CÂY CÂN BẰNG
Chương 4. CÂY TRÊN XÂU
Chương 5. CÁC CẤU TRÚC CÂY ĐẶC BIỆT

LỜI NÓI ĐẦU

Mở bất kì đề thi học sinh giỏi Tin học nào từ cấp tỉnh trở lên, em đều gặp những câu hỏi dạng: "Cho dãy n số và q truy vấn, mỗi truy vấn hoặc sửa một phần tử, hoặc hỏi tổng (hay giá trị lớn nhất) trên một đoạn". Với n và q tới $2 \cdot 10^5$, cách làm "mỗi truy vấn duyệt lại cả đoạn" cần khoảng $4 \cdot 10^{10}$ phép tính, chạy mất vài phút. Chìa khóa để vượt qua những bài như vậy là **cấu trúc dữ liệu dạng cây**: Tổ chức dữ liệu thành một cái cây sao cho mỗi thao tác chỉ phải đi qua một con đường từ gốc tới lá, dài khoảng $\log_2 n$ bước thay vì n bước.

Bộ tài liệu này giúp em làm chủ các cấu trúc cây một cách có hệ thống, từ chiếc heap đơn giản nhất mà chương trình nào cũng dùng, tới những cấu trúc chuyên sâu như cây phân đoạn bền vững, Segment Tree Beats, cây hậu tố, cây palindrome hay cây wavelet. Bộ sách gồm ba tập đi cùng nhau:

- **Tập 1: Lý thuyết** (cuốn sách này) gồm 22 bài học chia thành 5 chương: Các cấu trúc cây cơ bản; Cây phân đoạn nâng cao; Cây cân bằng; Cây trên xâu; Các cấu trúc cây đặc biệt.
- **Tập 2: Bài tập** gồm 66 bài, mỗi bài học có ba bài tập ở ba mức độ, có ràng buộc chia theo subtask như đề thi thật.
- **Tập 3: Hướng dẫn giải** phân tích từng bài tập từ cách ngây thơ tới cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm thực tế của từng cách làm.

Mỗi bài học không đưa cấu trúc dữ liệu ra ngay. Em sẽ được đặt trước một bài toán cụ thể, thử cách ngây thơ có sơ đồ khối, đếm số phép tính để thấy chỗ bế tắc, rồi được dẫn từng bước để tự "phát minh lại" cấu trúc cây giải quyết bế tắc đó. Mọi thao tác trên cây đều được chạy tay bằng hình vẽ, từng bước một; mọi chương trình trong sách đều đã được kiểm tra bằng cách đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên.

Chương trình được viết bằng C++ theo chuẩn C++14, chỉ dùng các thư viện chuẩn riêng lẻ (không dùng `bits/stdc++.h`) và đọc ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến. Tên biến được đặt ngắn gọn, thống nhất với công thức trong bài để em dễ đối chiếu.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường, giống như cách một người giỏi thật sự suy nghĩ khi gặp bài mới:

Phần	Nội dung
Khởi động	Một câu hỏi nhỏ, gần gũi để em bắt đầu nghĩ.
Bài toán	Đề bài cụ thể, có ví dụ và hình minh họa.
Ý tưởng ngây thơ	Cách làm đầu tiên ai cũng nghĩ ra, kèm sơ đồ khối và phân đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ chậm, và đặt câu hỏi dẫn tới ý tưởng mới.
Phá bế tắc	Từng bước: quan sát, dựng cấu trúc, chạy tay trên hình, chứng minh, chương trình.
Chương trình hoàn chỉnh	Chương trình đã kiểm thử, đọc ghi tệp đúng thể thức đề thi.
Mở rộng	Các biến thể và kĩ thuật liên quan thường gặp trong đề thi.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Ghi nhớ	Tóm tắt những điều cốt lõi nhất.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
BÀI TOÁN	Đề bài chính thức.
Ý TƯỞNG, CÔNG THỨC	Định nghĩa cấu trúc, công thức then chốt.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp, giải thích vì sao là O(n), O(log n), O(n log n), ...
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
VÌ SAO ĐÚNG?	Lập luận chứng minh tính đúng đắn.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Mức độ của các bài học được đánh dấu như sau: **Cơ bản** (chương 1) là những cấu trúc mà học sinh dự thi cấp tỉnh nào cũng cần thành thạo; **Nâng cao** (phần lớn chương 2, 3, 4) thường xuất hiện ở câu phân loại của đề cấp tỉnh và đề chọn đội tuyển; **Chuyên sâu** (Segment Tree Beats, cây palindrome, chương 5) dành cho học sinh hướng tới kì thi quốc gia và quốc tế.

Quy ước màu trong các hình vẽ được giữ thống nhất cho cả bộ sách: nút **vàng** là nút đang xét; nút **đỏ** là nút vi phạm hoặc vừa bị thay đổi; nút **xanh lá** là nút đã xử lí xong hoặc đóng góp vào đáp án; nút **xanh dương** là nút bình thường; nút **xám** là nút không phải thăm.

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Mỗi khi gặp một hình chạy tay, hãy che hình sau lại và tự vẽ trước. Mỗi khi gặp một cấu trúc mới, hãy tự trả lời bốn câu hỏi: Mỗi nút lưu gì? Hai nút con ghép lại thành nút cha thế nào? Một thao tác đi qua bao nhiêu nút? Vì sao số nút đó nhỏ? Chỉ khi tự viết lại được chương trình mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thanh Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **3**

CHƯƠNG 1. CÁC CẤU TRÚC CÂY CƠ BẢN **6**

Bài 1. Heap và hàng đợi ưu tiên 7
Bài 2. Cây Fenwick: Tổng tiền tố khi mảng liên tục thay đổi 24
Bài 3. Cây Fenwick nâng cao: Cập nhật đoạn, hai chiều, tìm phần tử thứ k 38
Bài 4. Cây phân đoạn: Truy vấn trên đoạn với phép kết hợp bất kì 51
Bài 5. Lan truyền lười: Cập nhật cả một đoạn trong O(log n) 66
Bài 6. Rừng hợp nhất DSU: Nén đường đi, hợp theo kích thước, có trọng số, quay lui 76
Bài 7. Cây nhị phân tìm kiếm, std::set và std::map 88

CHƯƠNG 2. CÂY PHÂN ĐOẠN NÂNG CAO **97**

Bài 8. Đi xuống trên cây phân đoạn và cây phân đoạn động 98
Bài 9. Cây phân đoạn bền vững 108
Bài 10. Cây phân đoạn trộn, cây phân đoạn hai chiều và cây lồng cây 118
Bài 11. Cây Li Chao: Giá trị nhỏ nhất của các đường thẳng 127
Bài 12. Segment Tree Beats: Cập nhật "cắt trần" trên đoạn 136

CHƯƠNG 3. CÂY CÂN BẰNG **146**

Bài 13. Treap và treap khóa ngầm định 147
Bài 14. Cây Splay 159
Bài 15. Cây AVL, cây đỏ đen và cây thứ tự trong PBDS 170

CHƯƠNG 4. CÂY TRÊN XÂU **182**

Bài 16. Trie, Trie nhị phân và Trie bền vững 183
Bài 17. Máy Aho-Corasick 191
Bài 18. Suffix Automaton và cây hậu tố 199
Bài 19. Cây palindrome (Eertree) 208

CHƯƠNG 5. CÁC CẤU TRÚC CÂY ĐẶC BIỆT **216**

Bài 20. Cây wavelet 217
Bài 21. Cây KD và cây tứ phân 225
Bài 22. Cây van Emde Boas và tập hợp 64 nhánh 234

Lê Thành Nam

CHƯƠNG 1. CÁC CẤU TRÚC CÂY CƠ BẢN

Một thư viện có một triệu cuốn sách xếp lộn xộn thì tìm một cuốn phải lục cả triệu ngăn. Nhưng nếu sách được chia theo tầng, mỗi tầng chia theo phòng, mỗi phòng chia theo giá, thì chỉ cần vài lần rẽ đúng hướng là tới nơi. Cấu trúc dữ liệu dạng cây làm đúng việc đó với dữ liệu: Chia nhỏ tập dữ liệu thành các nhóm lồng nhau, để mỗi thao tác chỉ đi qua một con đường ngắn từ gốc xuống lá.

Chương này xây nền móng cho cả cuốn sách với năm cấu trúc mà học sinh giỏi nào cũng phải viết được trong vài phút: heap, cây Fenwick, cây phân đoạn (có và không có lan truyền lười), rừng hợp nhất DSU và cây nhị phân tìm kiếm cùng các công cụ set, map của C++.

TRONG CHƯƠNG NÀY

Bài 1. Heap và hàng đợi ưu tiên: Luôn lấy được phần tử nhỏ nhất trong O(log n)

Bài 2. Cây Fenwick: Tổng tiền tố khi mảng liên tục thay đổi

Bài 3. Cây Fenwick nâng cao: Cập nhật đoạn, hai chiều, tìm phần tử thứ k

Bài 4. Cây phân đoạn: Truy vấn trên đoạn với phép kết hợp bất kì

Bài 5. Lan truyền lười: Cập nhật cả một đoạn trong O(log n)

Bài 6. Rừng hợp nhất DSU: Nén đường đi, hợp theo kích thước, DSU có trọng số, DSU quay lui

Bài 7. Cây nhị phân tìm kiếm, std::set và std::map

Lê Thành Nam

BÀI 1 Heap và hàng đợi ưu tiên

Học xong bài này, em sẽ:

- ✓ Hiểu vì sao "lấy phần tử nhỏ nhất của một tập luôn thay đổi" là việc khó nếu chỉ dùng mảng.
- ✓ Biết heap là một cây nhị phân hoàn chỉnh cất trong mảng, trong đó mỗi nút cha không lớn hơn các nút con.
- ✓ Tự cài đặt được hai thao tác vun lên và vun xuống, giải thích được vì sao mỗi thao tác chỉ tốn O(log n).
- ✓ Dùng thành thạo priority_queue của C++: heap lớn nhất, heap nhỏ nhất, heap chứa cặp số, so sánh tùy biến, xóa trễ.
- ✓ Nhận ra dấu hiệu dùng heap trong đề thi: "liên tục lấy ra phần tử lớn nhất (nhỏ nhất) trong một tập mà ta vẫn đang thêm phần tử vào".

KHỞI ĐỘNG

Bác thợ hàn có bốn đoạn dây dài 4, 3, 2 và 6 mét, cần nối thành một sợi duy nhất. Mỗi lần bác chọn hai đoạn bất kì và nối lại, tiền công bằng tổng độ dài hai đoạn được nối. Nối theo thứ tự nào thì tổng tiền công ít nhất?

Thử nối lần lượt theo thứ tự đã cho: 4 + 3 = 7, rồi 7 + 2 = 9, rồi 9 + 6 = 15; tổng tiền công là 7 + 9 + 15 = 31. Còn nếu mỗi lần luôn nối hai đoạn ngắn nhất: 2 + 3 = 5, rồi 4 + 5 = 9, rồi 6 + 9 = 15; tổng chỉ còn 29. Với bốn đoạn dây, em có thể thử bằng tay. Nhưng nếu có hai trăm nghìn đoạn dây, làm sao để "mỗi lần tìm hai đoạn ngắn nhất" thật nhanh?

BÀI TOÁN. NỐI DÂY [NOIDAY]

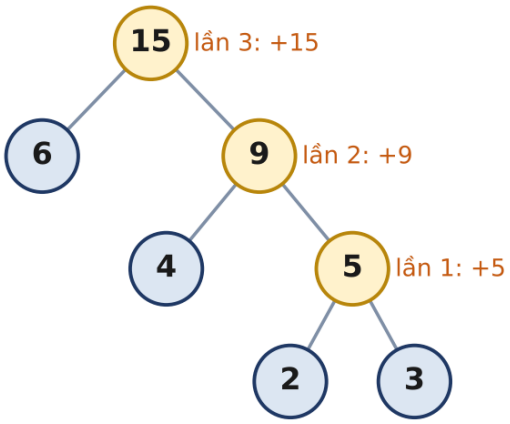
Có n đoạn dây với độ dài $a_1, a_2, ..., a_n$. Mỗi lần ta chọn hai đoạn dây bất kì có độ dài x và y, nối chúng thành một đoạn dài x + y và phải trả chi phí x + y. Hãy tìm tổng chi phí nhỏ nhất để nối tất cả thành một đoạn duy nhất.

Dữ liệu vào: Dòng đầu chứa số nguyên n ($1 \le n \le 2 \cdot 10^5$). Dòng thứ hai chứa n số nguyên $a_1, a_2, ..., a_n$ ($1 \le a_i \le 10^9$).

Kết quả: Một số nguyên là tổng chi phí nhỏ nhất.

Dữ liệu vào	Kết quả	Giải thích
4 4 3 2 6	29	Nối 2 với 3, rồi 4 với 5, rồi 6 với 9 (Hình 1.1).
3 5 5 5	25	Nối 5 với 5 tốn 10, rồi 10 với 5 tốn 15.
1 7	0	Chỉ có một đoạn, không phải nối.

Lê Thành Nam



Hình 1.1. *Cây ghép của ví dụ 1. Mỗi nút vàng là một lần nối; tổng chi phí 5 + 9 + 15 = 29*

Hình 13.6 cho thấy một cách nhìn thú vị: Quá trình nối dây tạo thành một cây, các đoạn dây ban đầu là lá, mỗi lần nối là một nút trong. Một đoạn dây nằm sâu bao nhiêu tầng thì độ dài của nó bị cộng vào chi phí bấy nhiêu lần (đoạn 2 và đoạn 3 nằm sâu 3 tầng nên bị tính 3 lần). Muốn chi phí nhỏ, các đoạn ngắn nên nằm sâu, các đoạn dài nên nằm nông. Đó chính là lí do của chiến lược "luôn nối hai đoạn ngắn nhất".

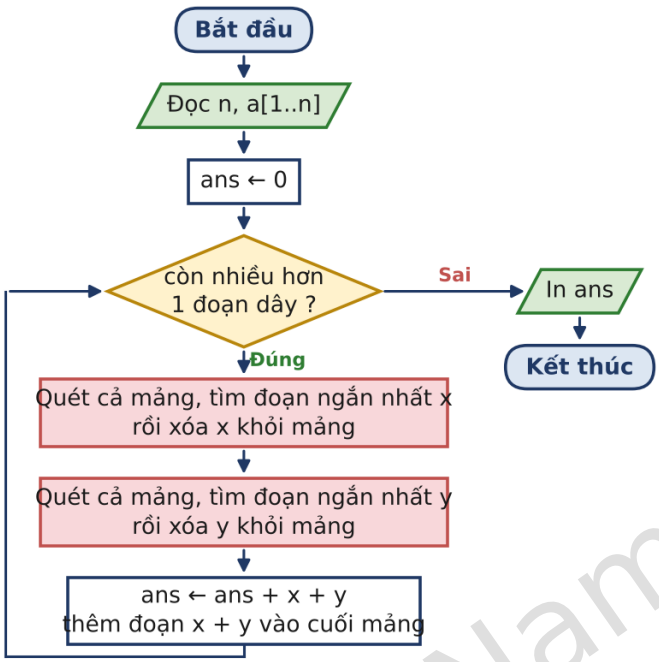
> **VÌ SAO NỐI HAI ĐOẠN NGẮN NHẤT LÀ TỐI ƯU?**
>
> Xét một cây ghép tối ưu bất kì. Gọi $x \le y$ là hai đoạn ngắn nhất. Ở tầng sâu nhất của cây có hai lá là anh em (cùng cha), gọi là p và q. Nếu đổi chỗ x với p và y với q, mỗi phép đổi chuyển đoạn ngắn hơn xuống sâu hơn và đoạn dài hơn lên nông hơn, nên chi phí không tăng (độ dài × độ sâu: đổi chỗ làm tổng thay đổi một lượng $(p - x)(\text{độ sâu của } x - \text{độ sâu của } p) \le 0$).
>
> Vậy luôn tồn tại một cách ghép tối ưu trong đó x và y được nối với nhau. Sau khi nối, đoạn x + y trở thành một đoạn mới và bài toán còn n − 1 đoạn, lặp lại lập luận. Đây chính là thuật toán Huffman nổi tiếng dùng để nén dữ liệu.

Thuật toán đã rõ. Phần còn lại của bài học trả lời câu hỏi: Làm sao **thực hiện** thuật toán đó đủ nhanh?

1. Ý tưởng ngây thơ: Mỗi lần quét cả mảng

Cách tự nhiên nhất là cất các đoạn dây trong một mảng. Mỗi lần cần nối, ta quét cả mảng để tìm đoạn ngắn nhất, xóa nó đi, quét lần nữa để tìm đoạn ngắn nhất tiếp theo, xóa nó đi, rồi thêm đoạn mới vào cuối mảng.

Lê Thanh Nam



Hình 1.2. *Sơ đồ khối của cách quét mảng. Hai ô đỏ là hai lần quét toàn bộ mảng*

Code ý chính: Quét mảng tìm hai đoạn ngắn nhất

```
while (a.size() > 1) {
    int p = 0;
    for (int i = 1; i < (int)a.size(); i++)
        if (a[i] < a[p]) p = i;
    long long x = a[p];
    a.erase(a.begin() + p);
    int q = 0;
    for (int i = 1; i < (int)a.size(); i++)
        if (a[i] < a[q]) q = i;
    long long y = a[q];
    a.erase(a.begin() + q);
    ans += x + y;
    a.push_back(x + y);
}
```

Chạy tay với ví dụ 1: Mảng ban đầu [4, 3, 2, 6]. Lần 1 tìm được x = 2, y = 3, mảng thành [4, 6, 5], ans = 5. Lần 2 tìm được x = 4, y = 5, mảng thành [6, 9], ans = 14. Lần 3 tìm được x = 6, y = 9, mảng thành [15], ans = 29. Kết quả đúng.

VÌ SAO CÁCH NGÂY THƠ CHẬM? ĐẾM TỪNG BƯỚC

- Vòng lặp ngoài chạy $n - 1$ lần, vì mỗi lần nối làm số đoạn giảm đi 1.
- Ở lần thứ k, mảng còn $n - k + 1$ phần tử. Hai lần quét và hai lần xóa (xóa giữa mảng phải dồn các phần tử phía sau) tốn khoảng $4(n - k)$ phép tính.
- Tổng cộng: $4(n - 1) + 4(n - 2) + \ldots + 4 \cdot 1 = 2n(n - 1)$, tức là $O(n^2)$.
- Với $n = 2 \cdot 10^5$: khoảng $8 \cdot 10^{10}$ phép tính, cần vài phút, trong khi giới hạn là 1 giây.

Lê Thành Nam

BẾ TẮC

Mỗi lần nối, ta quét **toàn bộ** mảng chỉ để lấy ra **một** phần tử nhỏ nhất, rồi vứt bỏ hết mọi thông tin về thứ tự của những phần tử còn lại. Lần sau lại quét từ đầu. Liệu có cách nào "ghi nhớ" một phần thứ tự, để lần sau tìm phần tử nhỏ nhất mà không phải nhìn lại tất cả?

2. Phá bế tắc: Dựng heap từng bước

Bước 1 Hai cực đoan, và một con đường ở giữa

Ta cần một cấu trúc hỗ trợ ba thao tác: **thêm** một phần tử, **xem** phần tử nhỏ nhất, **lấy ra** phần tử nhỏ nhất. Hãy so sánh hai cách cất dữ liệu quen thuộc:

Cách cất	Thêm	Xem nhỏ nhất	Lấy ra nhỏ nhất
Mảng không sắp xếp	O(1)	O(n)	O(n)
Mảng luôn được sắp xếp	O(n)	O(1)	O(1)
Heap (bài này)	O(log n)	O(1)	O(log n)

Mảng không sắp xếp thì quá "lười": không giữ chút thứ tự nào. Mảng sắp xếp thì quá "chăm": giữ thứ tự của **mọi** cặp phần tử, trong khi ta chỉ cần biết phần tử nhỏ nhất. Ta cần một cấu trúc ở giữa: giữ **một phần** thứ tự, vừa đủ để biết ai nhỏ nhất, và rẻ để sửa khi có thay đổi.

Hãy nghĩ tới sơ đồ tổ chức của một trường học: hiệu trưởng ở trên cùng, dưới là các tổ trưởng, dưới nữa là giáo viên. Mỗi người chỉ cần "đứng trên" cấp dưới trực tiếp của mình, không cần so với người ở nhánh khác. Thế mà người đứng đầu vẫn là người "cao nhất" toàn trường. Heap chính là sơ đồ tổ chức như vậy.

Bước 2 Cây nhị phân hoàn chỉnh cất trong mảng

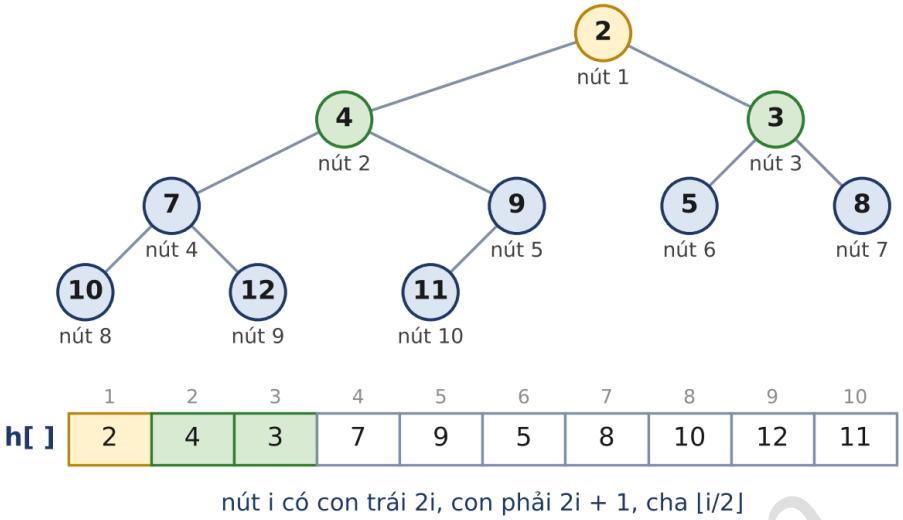
Một **cây nhị phân hoàn chỉnh** là cây nhị phân trong đó mọi tầng đều đầy, trừ tầng cuối cùng được lấp dần từ trái sang phải. Đánh số các nút từ trên xuống dưới, từ trái sang phải bắt đầu từ 1, ta có một quy luật rất đẹp:

ĐÁNH SỐ NÚT TRONG CÂY NHỊ PHÂN HOÀN CHỈNH

Nút i có con trái là nút **2i**, con phải là nút **2i + 1**, và cha là nút **⌊i/2⌋** (i > 1).

Vì vậy ta không cần con trỏ: Cả cây được cất trong một mảng h[1..n], nút i chính là ô h[i].

Lê Thành Nam



Hình 1.3. *Cây nhị phân hoàn chỉnh 10 nút và mảng tương ứng. Con của nút 1 là nút 2, 3; con của nút 4 là nút 8, 9*

Kiểm tra trên Hình 1.3: nút 4 (giá trị 7) có con là nút 8 và 9 (giá trị 10 và 12); nút 5 (giá trị 9) có con trái là nút 10 (giá trị 11), còn con phải 11 không tồn tại vì 11 > n = 10. Cha của nút 9 là $\lfloor 9/2 \rfloor = 4$.

Vì mỗi tầng chứa gấp đôi tầng trên, cây n nút có chiều cao chỉ $\lfloor \log_2 n \rfloor$: với $n = 2 \cdot 10^5$, cây cao 17 tầng. Mọi thao tác chỉ đi dọc một con đường từ gốc tới lá đều tốn không quá 18 bước. Đây là "phép màu" của mọi cấu trúc cây trong cuốn sách này.

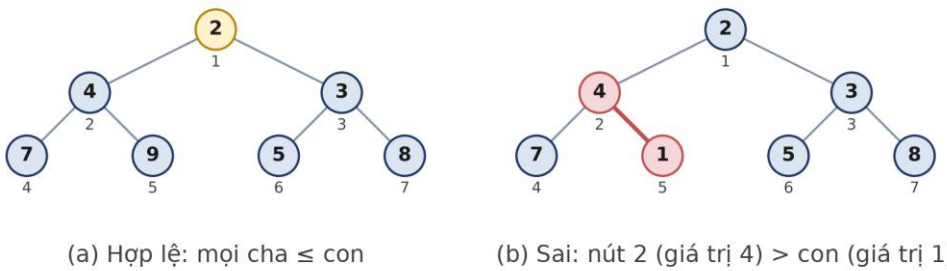
Bước 3 Tính chất heap: Cha không lớn hơn con

ĐỊNH NGHĨA HEAP (MIN-HEAP)

Một **heap nhỏ nhất** (min-heap) là một cây nhị phân hoàn chỉnh, trong đó giá trị ở mỗi nút **không lớn hơn** giá trị ở các nút con của nó: $h[\lfloor i/2 \rfloor] \le h[i]$ với mọi $i \ge 2$.

Đổi chiều bất đẳng thức, ta được **heap lớn nhất** (max-heap).

Từ định nghĩa suy ra ngay: đi từ bất kì nút nào ngược lên gốc, các giá trị không tăng. Vì vậy **gốc h[1] là phần tử nhỏ nhất của cả heap**. Thao tác "xem nhỏ nhất" chỉ là đọc h[1], tốn O(1).



Hình 1.4. *Heap hợp lệ và không hợp lệ. Chỉ cần một cặp cha con sai thứ tự là cây không còn là heap*

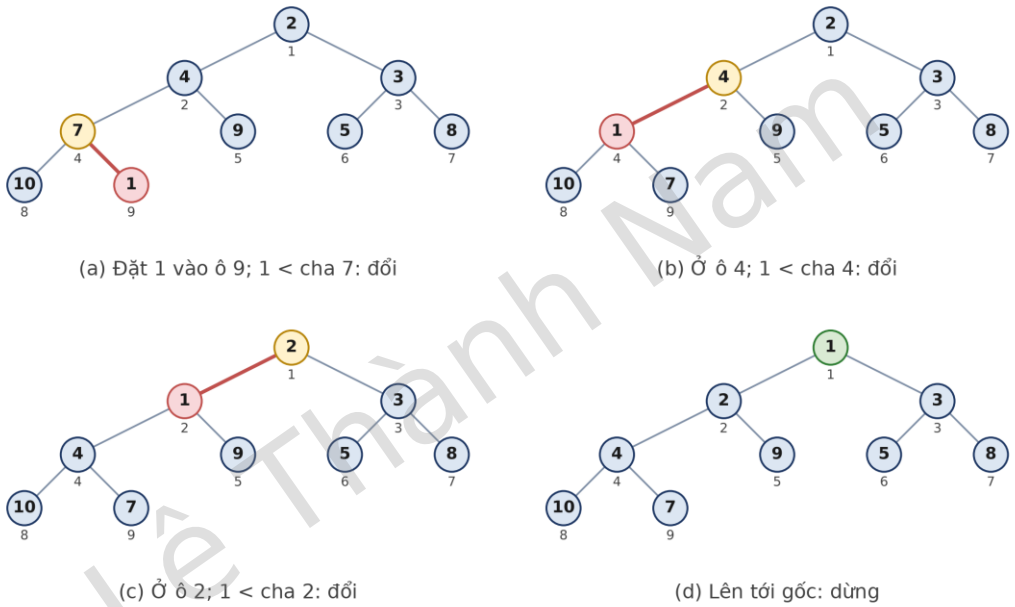
Lê Thành Nam

Hãy để ý: heap **không** sắp xếp toàn bộ. Ở Hình 1.4(a), nút 3 (giá trị 3) nhỏ hơn nút 2 (giá trị 4) dù nằm bên phải; nút 4 (giá trị 7) lớn hơn nút 6 (giá trị 5) dù ở tầng trên. Heap chỉ đòi hỏi thứ tự **dọc** theo các cạnh cha con. Chính vì đòi hỏi ít, nó mới rẻ để duy trì.

Bước 4 Thêm phần tử: Đặt vào cuối, rồi vun lên

Muốn thêm giá trị x mà cây vẫn hoàn chỉnh, chỗ duy nhất có thể đặt là ô cuối cùng h[n + 1]. Nhưng x có thể nhỏ hơn cha của nó, phá vỡ tính chất heap. Cách sửa rất tự nhiên: **so x với cha, nếu x nhỏ hơn thì đổi chỗ**, rồi tiếp tục so với cha mới, cho tới khi x không nhỏ hơn cha hoặc x lên tới gốc. Thao tác này gọi là **vun lên** (sift-up).

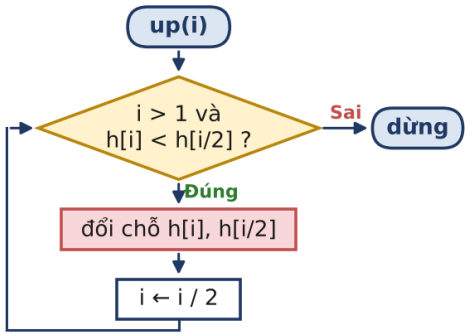
Hãy theo dõi việc thêm giá trị 1 vào heap [2, 4, 3, 7, 9, 5, 8, 10]:



Hình 1.5. *Thêm 1 vào heap: Mỗi bước 1 đổi chỗ với cha cho tới khi lên tới gốc*

Bước	Vị trí của 1	Việc làm
(a)	ô 9, cha là ô 4 (giá trị 7)	1 < 7 nên đổi chỗ.
(b)	ô 4, cha là ô 2 (giá trị 4)	1 < 4 nên đổi chỗ.
(c)	ô 2, cha là ô 1 (giá trị 2)	1 < 2 nên đổi chỗ.
(d)	ô 1 (gốc)	Không còn cha: dừng. Mảng thành [1, 2, 3, 4, 9, 5, 8, 10, 7].

Lê Thành Nam



Hình 1.6. *Sơ đồ khối thao tác vun lên up(i)*

Code ý chính: Thêm phần tử và vun lên

```
void up(int i) {
    while (i > 1 && h[i] < h[i / 2]) {
        swap(h[i], h[i / 2]);
        i /= 2;
    }
}
void push(long long x) {
    h[++sz] = x;
    up(sz);
}
```

VÌ SAO VUN LÊN GIỮ ĐÚNG TÍNH CHẤT HEAP?

Trước khi thêm, cây là heap. Khi x đang ở ô i và x < h[cha], ta đổi chỗ. Giá trị cũ của cha (gọi là p) xuống ô i: p vẫn ≤ các con cũ của ô i (vì trước đó p là tổ tiên của chúng). Còn x lên ô cha: x < p ≤ anh em của ô i, nên x ≤ cả hai con mới. Chỉ còn một cặp có thể sai là x với cha mới của nó, và vòng lặp tiếp tục kiểm tra đúng cặp đó.

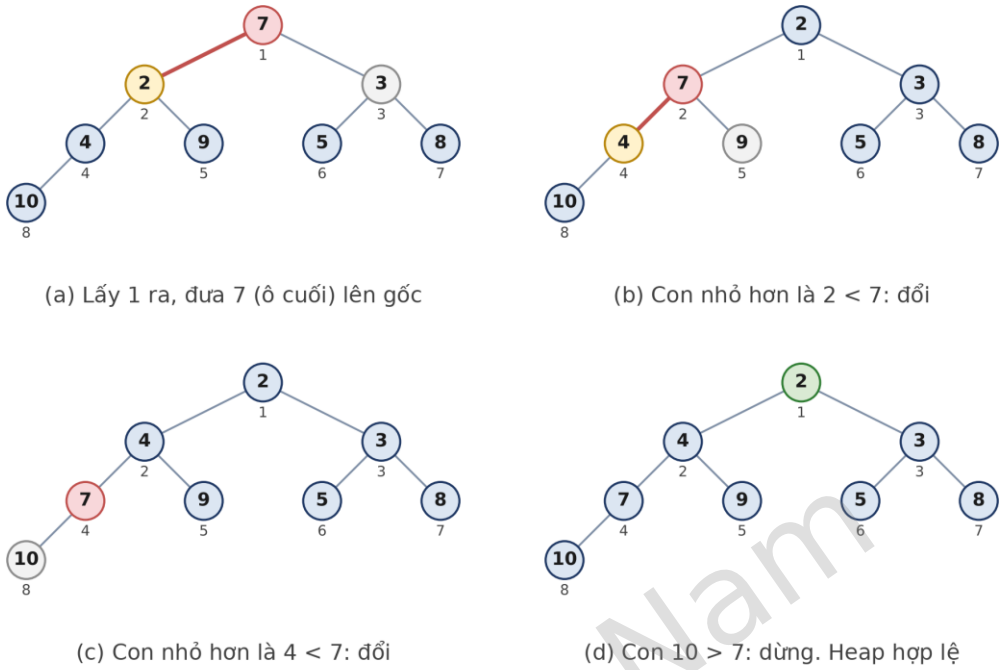
Số phép tính: Mỗi lần đổi chỗ, x lên một tầng; cây cao $\lfloor \log_2 n \rfloor$ nên vun lên tốn **O(log n)**.

Lê Thành Nam

Bước 5 Lấy ra phần tử nhỏ nhất: Đưa phần tử cuối lên gốc, rồi vun xuống

Phần tử nhỏ nhất nằm ở gốc. Sau khi lấy nó đi, gốc bị trống. Để cây vẫn hoàn chỉnh, ta lấy phần tử ở **ô cuối cùng** lên lấp vào gốc và giảm n đi 1. Lúc này giá trị ở gốc có thể lớn hơn con của nó. Cách sửa: **so với con nhỏ hơn trong hai con, nếu lớn hơn con đó thì đổi chỗ**, rồi tiếp tục đi xuống. Thao tác này gọi là **vun xuống** (sift-down).

Hãy lấy phần tử nhỏ nhất (giá trị 1) khỏi heap vừa có ở Bước 4:

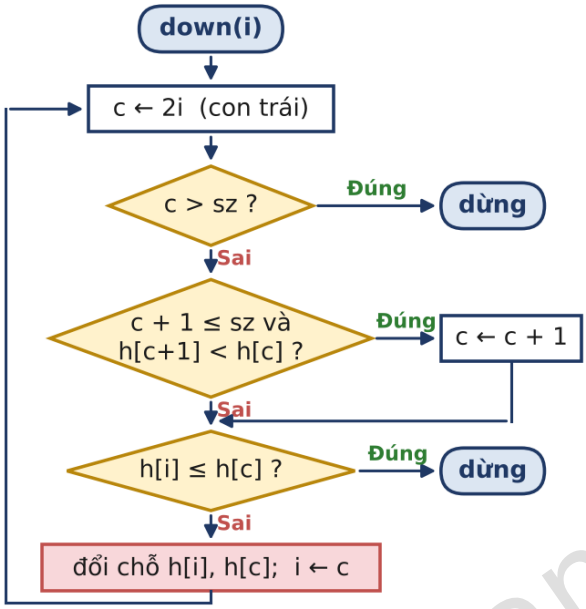


Hình 1.7. *Lấy ra phần tử nhỏ nhất: Phần tử cuối (7) lên gốc rồi chìm dần xuống*

Bước	Vị trí của 7	Việc làm
(a)	ô 1, hai con là 2 và 3	Con nhỏ hơn là 2; 7 > 2 nên đổi chỗ với ô 2.
(b)	ô 2, hai con là 4 và 9	Con nhỏ hơn là 4; 7 > 4 nên đổi chỗ với ô 4.
(c)	ô 4, chỉ có một con là 10	7 ≤ 10: dừng.
(d)	| Mảng thành [2, 4, 3, 7, 9, 5, 8, 10]: đúng heap ban đầu.	

Vì sao phải chọn con **nhỏ hơn**? Ở bước (a), nếu đổi 7 với con lớn hơn là 3, thì 3 lên gốc và trở thành cha của 2, sai ngay tính chất heap. Đổi với con nhỏ hơn thì phần tử mới lên làm cha vẫn ≤ anh em của nó.

Lê Thành Nam



Hình 1.8. *Sơ đồ khối thao tác vun xuống down(i)*

Code ý chính: Lấy ra phần tử nhỏ nhất và vun xuống

```
void down(int i) {
    while (true) {
        int c = 2 * i;
        if (c > sz) break;
        if (c + 1 <= sz && h[c + 1] < h[c]) c++;
        if (h[i] <= h[c]) break;
        swap(h[i], h[c]);
        i = c;
    }
}
long long pop() {
    long long r = h[1];
    h[1] = h[sz--];
    down(1);
    return r;
}
```

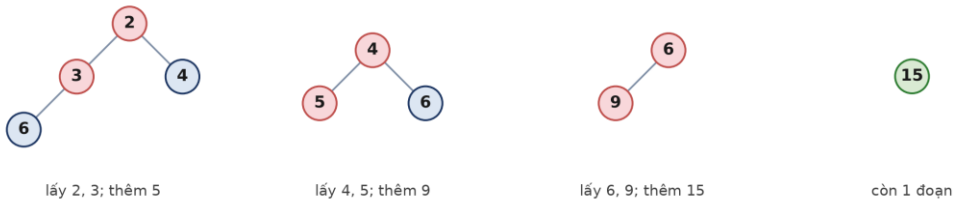
ĐẾM SỐ PHÉP TÍNH CỦA VUN XUỐNG

Mỗi vòng lặp làm 2 phép so sánh và tối đa 1 phép đổi chỗ, rồi đi xuống 1 tầng. Cây cao $\lfloor \log_2 n \rfloor$ nên vun xuống tốn **O(log n)**. Với $n = 2 \cdot 10^5$, mỗi lần lấy ra chỉ tốn khoảng 17 vòng lặp, thay vì quét $2 \cdot 10^5$ phần tử.

Bước 6 Ghép lại: Giải bài NỐI DÂY bằng heap

Thay hai lần quét mảng bằng hai lần `pop()`, và thay việc thêm vào cuối mảng bằng `push()`:

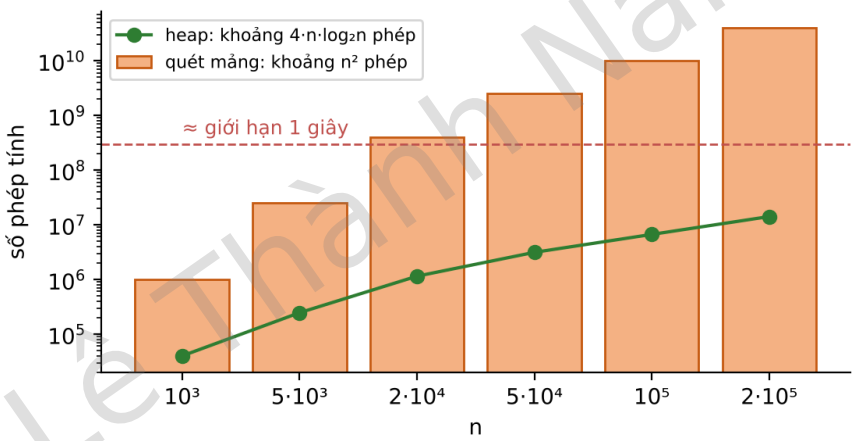
Lê Thành Nam



Hình 1.9. *Trạng thái heap qua ba lần nối của ví dụ 1. Hai nút đỏ là hai đoạn sắp được lấy ra*

Lần	Heap trước khi nối	Lấy ra	Thêm vào	ans
1	[2, 3, 4, 6]	2 và 3	5	5
2	[4, 5, 6]	4 và 5	9	14
3	[6, 9]	6 và 9	15	29

Mỗi lần nối gồm 2 lần lấy ra và 1 lần thêm vào, mỗi lần O(log n). Có n − 1 lần nối, cộng với n lần thêm ban đầu, nên tổng độ phức tạp là **O(n log n)**. Với $n = 2 \cdot 10^5$, chỉ khoảng $2 \cdot 10^5 \times 4 \times 18 \approx 1{,}4 \cdot 10^7$ phép tính, chạy trong vài chục mili giây.



Hình 1.10. *Số phép tính của cách quét mảng (cột cam) và của heap (đường xanh), trục đứng theo thang lôgarit*

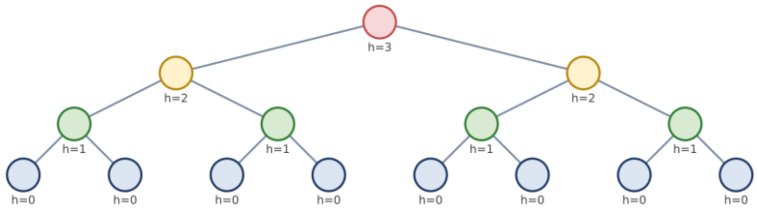
Bước 7 Xây heap từ mảng có sẵn trong O(n)

Khi đã có sẵn n phần tử, thay vì push từng phần tử (O(n log n)), ta có thể "vun" cả mảng một lần: gọi `down(i)` với i chạy từ ⌊n/2⌋ **lùi về** 1. Lúc gọi down(i), hai cây con của nút i đều đã là heap (vì chúng có chỉ số lớn hơn, đã được xử lí trước), nên down(i) biến cả cây con gốc i thành heap.

Code ý chính: Vun đống cả mảng (heapify)

```
for (int i = sz / 2; i >= 1; i--) down(i);
```

Lê Thành Nam



Hình 1.11. *Cây 15 nút: 8 nút có độ cao 0, 4 nút độ cao 1, 2 nút độ cao 2, 1 nút độ cao 3*

VÌ SAO VUN ĐỐNG CẢ MẢNG CHỈ TỐN O(N)?

- down(i) tốn số bước bằng **độ cao** của nút i (khoảng cách từ i tới lá sâu nhất bên dưới), không phải độ sâu của cây.
- Trong cây n nút, có khoảng n/2 nút độ cao 0, n/4 nút độ cao 1, n/8 nút độ cao 2, ...
- Tổng số bước ≤ n·(0/2 + 1/4 + 2/8 + 3/16 + ...) = n·1 = n. (Tổng 1/4 + 2/8 + 3/16 + ... bằng đúng 1.)
- Với Hình 1.11: 4·1 + 2·2 + 1·3 = 11 bước, nhỏ hơn n = 15.

Nhận xét này cho ta thuật toán sắp xếp vun đống (heap sort): vun đống O(n), rồi pop n lần O(n log n).

3. Hàng đợi ưu tiên trong C++

Trong phòng thi, ta hiếm khi tự viết heap mà dùng `priority_queue` có sẵn trong thư viện `<queue>`. Hiểu cách heap làm việc giúp em dùng nó đúng và biết chính xác mỗi thao tác tốn bao nhiêu.

Câu lệnh	Ý nghĩa	Độ phức tạp
`priority_queue<int> q;`	heap **lớn nhất** (mặc định)	O(1)
`q.push(x);`	thêm x	O(log n)
`q.top()`	xem phần tử ưu tiên nhất	O(1)
`q.pop();`	xóa phần tử ưu tiên nhất (không trả về giá trị)	O(log n)
`q.size(), q.empty()`	số phần tử, kiểm tra rỗng	O(1)

Heap nhỏ nhất

Mặc định `priority_queue` là heap **lớn nhất**. Muốn heap nhỏ nhất, ta khai báo thêm kiểu vector và phép so sánh `greater` (thư viện `<functional>`):

Code ý chính: Heap nhỏ nhất

```
priority_queue<long long, vector<long long>, greater<long long> > q;
```

Một mẹo nhanh khác là cất số đối −x vào heap lớn nhất, khi lấy ra thì đổi dấu lại. Cách này gọn nhưng dễ quên đổi dấu; trong sách ta dùng `greater` cho rõ ràng.

Lê Thành Nam

Heap chứa cặp số

Rất thường gặp: ta muốn lấy phần tử nhỏ nhất theo một khóa, nhưng cần biết thêm "nó là ai". Kiểu `pair` được so sánh theo thành phần thứ nhất trước, bằng nhau mới xét thành phần thứ hai, nên ta đặt khóa ở vị trí đầu:

Code ý chính: Heap các cặp (khóa, chỉ số)

```
typedef pair<long long, int> P;
priority_queue<P, vector<P>, greater<P> > q;
q.push(P(a[i] + b[0], i));
P x = q.top();
q.pop();
long long key = x.first;
int i = x.second;
```

So sánh tùy biến

Với dữ liệu nhiều trường, ta viết một `struct` và định nghĩa phép so sánh. Hãy nhớ quy tắc ngược đời của `priority_queue`: phần tử ở `top()` là phần tử **"lớn nhất"** theo phép so sánh. Vì vậy, muốn ưu tiên khoảng cách nhỏ, ta phải viết phép so sánh "ngược":

Code ý chính: Ưu tiên d nhỏ, d bằng nhau thì ưu tiên id nhỏ

```
struct Item {
    long long d;
    int id;
    bool operator<(const Item& o) const {
        if (d != o.d) return d > o.d;
        return id > o.id;
    }
};
priority_queue<Item> q;
```

Xóa trễ (lazy deletion)

`priority_queue` không cho xóa một phần tử bất kì. Khi cần xóa x, ta không xóa ngay mà ghi nhận "x đã bị xóa" (đếm số lần xóa của x bằng một `map`, hoặc dùng một heap thứ hai chứa các phần tử đã xóa). Mỗi khi cần `top()`, nếu phần tử ở đỉnh là phần tử đã bị xóa thì pop nó đi và giảm bộ đếm. Mỗi phần tử bị pop tối đa một lần, nên tổng chi phí vẫn là O(log n) cho mỗi thao tác tính trung bình.

Code ý chính: Heap lớn nhất có xóa trễ bằng heap thứ hai

```
priority_queue<int> q, del;
void erase(int x) { del.push(x); }
int top() {
    while (!del.empty() && q.top() == del.top()) {
        q.pop();
        del.pop();
    }
    return q.top();
}
```

Lê Thành Nam

4. Chương trình hoàn chỉnh

Phần đọc tệp được viết theo kiểu "có tệp thì đọc tệp": Khi chấm bằng Themis với tệp NOIDAY.INP, chương trình đọc và ghi tệp; khi nộp lên trang chấm trực tuyến (đọc từ bàn phím), điều kiện `fopen` sai nên chương trình đọc chuẩn như bình thường.

Lê Thành Nam

NOIDAY.CPP: Heap tự cài đặt, O(n log n)

```cpp
#include <cstdio>
#include <iostream>
#include <algorithm>
using namespace std;

const int N = 200005;
long long h[N];
int sz = 0;

void up(int i) {
    while (i > 1 && h[i] < h[i / 2]) {
        swap(h[i], h[i / 2]);
        i /= 2;
    }
}

void down(int i) {
    while (true) {
        int c = 2 * i;
        if (c > sz) break;
        if (c + 1 <= sz && h[c + 1] < h[c]) c++;
        if (h[i] <= h[c]) break;
        swap(h[i], h[c]);
        i = c;
    }
}

void push(long long x) {
    h[++sz] = x;
    up(sz);
}

long long pop() {
    long long r = h[1];
    h[1] = h[sz--];
    down(1);
    return r;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("NOIDAY.INP", "r")) {
        freopen("NOIDAY.INP", "r", stdin);
        freopen("NOIDAY.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    for (int i = 0; i < n; i++) {
        long long x;
        cin >> x;
        push(x);
    }
    long long ans = 0;
    while (sz > 1) {
        long long a = pop();
        long long b = pop();
        ans += a + b;
        push(a + b);
    }
    cout << ans << '\n';
    return 0;
}
```

Lê Thành Nam

NOIDAY.CPP: Dùng priority_queue, O(n log n)

```cpp
#include <cstdio>
#include <iostream>
#include <queue>
#include <vector>
#include <functional>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("NOIDAY.INP", "r")) {
        freopen("NOIDAY.INP", "r", stdin);
        freopen("NOIDAY.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    priority_queue<long long, vector<long long>, greater<long long> > q;
    for (int i = 0; i < n; i++) {
        long long x;
        cin >> x;
        q.push(x);
    }
    long long ans = 0;
    while (q.size() > 1) {
        long long a = q.top(); q.pop();
        long long b = q.top(); q.pop();
        ans += a + b;
        q.push(a + b);
    }
    cout << ans << '\n';
    return 0;
}
```

Lê Thành Nam

Vài chi tiết đáng chú ý:

- Tổng chi phí có thể tới khoảng $2 \cdot 10^5 \times 10^9 \times 18 \approx 3{,}6 \cdot 10^{15}$, vượt xa giới hạn của `int`. Mọi giá trị trong heap đều phải là `long long`.
- Khi n = 1, vòng `while` không chạy lần nào và chương trình in 0, đúng yêu cầu.
- Hai chương trình cho kết quả như nhau trên mọi bộ test; chương trình dùng `priority_queue` ngắn hơn và là lựa chọn mặc định trong phòng thi.

5. Mở rộng

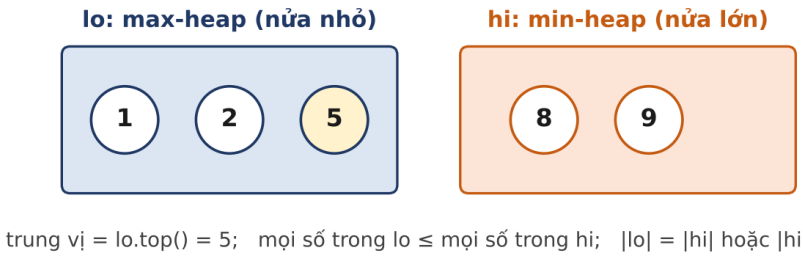
Mở rộng 1: k phần tử lớn nhất của một luồng dữ liệu

Đọc lần lượt n số, cần biết k số lớn nhất (k nhỏ hơn rất nhiều so với n). Giữ một heap **nhỏ nhất** có tối đa k phần tử: thêm số mới vào, nếu heap vượt quá k phần tử thì pop phần tử nhỏ nhất. Cuối cùng heap chứa đúng k số lớn nhất, và đỉnh heap là số lớn thứ k. Độ phức tạp O(n log k), bộ nhớ chỉ O(k).

Mở rộng 2: Trung vị động bằng hai heap

Các số đến lần lượt, sau mỗi số cần biết trung vị. Chia các số đã có thành hai nửa: nửa nhỏ cất trong heap lớn nhất `lo`, nửa lớn cất trong heap nhỏ nhất `hi`, sao cho mọi số trong `lo` không lớn hơn mọi số trong `hi` và |lo| bằng |hi| hoặc |hi| + 1. Khi đó trung vị chính là `lo.top()`. Mỗi

lần thêm số, ta đưa nó vào nửa phù hợp rồi chuyển một phần tử qua lại để cân bằng kích thước. Đây là Bài tập 1.1.



Hình 1.12. *Hai heap giữ hai nửa của dãy {1, 2, 5, 8, 9}; trung vị là đỉnh của heap nửa nhỏ*

Mở rộng 3: Trộn k dãy đã sắp xếp và "biên" của bài toán

Muốn trộn k dãy đã sắp xếp, ta cho phần tử đầu của mỗi dãy vào heap; mỗi lần lấy ra phần tử nhỏ nhất thì đưa phần tử kế tiếp **của cùng dãy đó** vào. Heap luôn chỉ chứa k phần tử là "biên" của quá trình. Ý tưởng "heap chứa biên" này xuất hiện ở rất nhiều nơi: thuật toán Dijkstra (biên là các đỉnh đã có khoảng cách tạm), bài tìm k tổng nhỏ nhất (Bài tập 1.2), bài tìm k tập con có tổng nhỏ nhất.

Mở rộng 4: Tham lam "hối hận" với heap

Một kĩ thuật rất mạnh: duyệt các lựa chọn theo một thứ tự, cứ nhận tạm lựa chọn hiện tại; nếu vi phạm ràng buộc thì "hối hận", bỏ đi lựa chọn kém nhất đã nhận, mà lựa chọn kém nhất lấy ra từ heap trong O(log n). Bài tập 1.3 (CÔNG VIỆC) là ví dụ điển hình.

Mở rộng 5: Heap d nhánh và heap trộn được

Nếu mỗi nút có d con thay vì 2, cây thấp hơn ($\log_d n$ tầng): vun lên nhanh hơn nhưng vun xuống chậm hơn (mỗi tầng so sánh d con). Ngoài ra còn có các heap trộn được (leftist heap, skew heap, pairing heap) cho phép trộn hai heap trong O(log n); chúng dùng trong kĩ thuật trộn từ nhỏ vào lớn trên cây.

BẪY THƯỜNG GẶP

- `priority_queue` mặc định là heap **lớn nhất**. Quên `greater` là lỗi phổ biến nhất.
- `pop()` không trả về giá trị: phải gọi `top()` trước rồi mới `pop()`.
- Gọi `top()` hoặc `pop()` khi heap rỗng là lỗi chạy chương trình. Luôn kiểm tra `empty()`.
- Phép so sánh tùy biến bị ngược chiều: phần tử ở đỉnh là phần tử "lớn nhất" theo `operator<`.
- Tràn số: tổng các giá trị trong bài nối dây vượt `int`. Dùng `long long` cho cả phần tử trong heap.
- Tự cài heap: trong vun xuống quên kiểm tra con phải có tồn tại hay không ($c + 1 \le sz$).

Lê Thành Nam

GHI NHỚ

- Heap là cây nhị phân hoàn chỉnh cất trong mảng: con của i là 2i, 2i + 1, cha là ⌊i/2⌋. Chiều cao ⌊log₂n⌋.
- Tính chất heap chỉ ràng buộc thứ tự cha con, đủ để gốc là phần tử nhỏ nhất (lớn nhất).
- Thêm: đặt cuối mảng rồi vun lên. Lấy ra: đưa phần tử cuối lên gốc rồi vun xuống (đổi với con nhỏ hơn). Cả hai đều O(log n).
- Vun đống cả mảng từ ⌊n/2⌋ lùi về 1 chỉ tốn O(n).
- Dấu hiệu dùng heap: liên tục lấy ra phần tử nhỏ nhất (lớn nhất) của một tập đang được thêm phần tử.

LUYỆN TẬP

Bài	Tên	Gợi ý liên hệ
1.1	TRUNG VỊ	Sau mỗi số mới, in trung vị của các số đã có. Hai heap giữ hai nửa (Mở rộng 2).
1.2	TỔNG NHỎ	In k tổng $a_i + b_j$ nhỏ nhất. Heap chứa "biên": mỗi a_i ghép với một b_j tiếp theo (Mở rộng 3).
1.3	CÔNG VIỆC	Chọn các công việc có hạn chót để được nhiều tiền nhất. Tham lam "hối hận" bằng heap (Mở rộng 4).

Tự kiểm tra nhanh

1. Thêm lần lượt 5, 3, 8, 1, 4 vào một heap nhỏ nhất rỗng. Mảng h sau cùng là gì? (Đáp số: [1, 3, 8, 5, 4].)
2. Từ heap ở câu 1, lấy ra phần tử nhỏ nhất một lần. Mảng h còn lại là gì? (Đáp số: [3, 4, 8, 5].)
3. Tính tổng chi phí nhỏ nhất để nối các đoạn dây 1, 1, 1, 1, 1. (Đáp số: 12. Nối 1 + 1 = 2, 1 + 1 = 2, 1 + 2 = 3, 2 + 3 = 5; tổng 2 + 2 + 3 + 5 = 12.)
4. Một heap lớn nhất có 100 phần tử. Phần tử nhỏ nhất có thể nằm ở những ô nào? (Đáp số: chỉ có thể ở một lá, tức các ô 51 tới 100.)

Lê Thành Nam